

ADAMAWA STATE UNIVERSITY MUBI (ADSU)

FACULTY OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

REG NUMBER: 23D/360091

COURSE CODE: CSC 403

COURSES TITLE: INTRODUCTION TO SOFTWARE ENGINEERING

ASSIGNMENT

1. Meaning of Software Life Cycle

The Software Development Life Cycle (SDLC) refers to a structured process used by software developers to design, develop, test, and maintain software systems. It provides a systematic approach that ensures the production of high-quality software that meets user requirements and is delivered within time and budget constraints. According to Pressman (2014), SDLC is a framework that defines activities, actions, and tasks required to build and maintain software systems. The main purpose of the software life cycle is to provide guidance and control throughout the development process. It helps organizations manage risks, improve productivity, and ensure customer satisfaction. By following the SDLC, developers can reduce errors, improve documentation, and enhance system reliability.

Stages of Software Life Cycle

The software life cycle consists of several important stages. These stages include:

1. Requirements Analysis: This stage involves gathering and analyzing user needs and system requirements.
2. System Design: In this stage, the overall system architecture and design are developed.
3. Implementation (Coding): Developers write program codes based on the design specifications.
4. Testing: The software is tested to identify and correct errors and defects.
5. Deployment: The system is installed and delivered to users for operation.
6. Maintenance: This involves updating, improving,

and fixing the system after deployment (Sommerville, 2016). Each stage plays a vital role in ensuring that the final software product is efficient, reliable, and user-friendly.

2. Software Life Cycle Models

Software life cycle models are structured approaches that define how the stages of software development are organized and executed. They provide guidelines for planning, managing, and controlling the development process. These models help developers understand their roles and responsibilities at each stage of development. Different models are used depending on project size, complexity, and requirements. Each model has its advantages and disadvantages, and selecting the right model is essential for project success.

3. Explanation of Software Life Cycle Models

Several software development models are commonly used in practice. These include:

- a. Waterfall Model: The Waterfall model follows a linear and sequential approach where each phase must be completed before moving to the next. It is simple and easy to manage but lacks flexibility (Pressman, 2014).
- b. Incremental Model: This model develops software in small parts called increments. Each increment adds new features to the system. It allows early delivery and continuous improvement.
- c. Spiral Model: The Spiral model combines iterative development with risk analysis. It emphasizes identifying and reducing risks at every stage. This model is suitable for large and complex projects (Sommerville, 2016).
- d. Agile Model: Agile focuses on flexibility, teamwork, and customer collaboration. Development is done in short cycles called iterations. It allows quick response to changes and continuous feedback.
- e. V-Model: The V-Model emphasizes verification and validation. Each development phase is linked with a corresponding testing phase, ensuring high-quality output.

These models help organizations manage time, cost, and quality effectively.

Conclusion

The Software Development Life Cycle is an essential framework for building reliable and efficient software systems. It provides a systematic approach that guides developers through planning, design, implementation, testing, and maintenance. Understanding different life cycle models enables organizations to select appropriate methods for their projects. By following proper SDLC practices, developers can improve software quality, reduce risks, and achieve user satisfaction.

References

Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education Limited.